

SENTINEL

Sistema de Optimización de Patrullaje en Santa Cruz



Integrantes:

Diego Andrade

Daniel Roca

Luis Marcelo Aguilera

Ana Cabrera

| Problema

Las unidades de patrullaje deben desplazarse constantemente entre diferentes puntos de la ciudad para atender incidentes y realizar recorridos preventivos

Sin una planificación adecuada pueden ocurrir problemas como:

- Selección de rutas poco eficientes
- Mayor tiempo de respuesta ante emergencias
- Mayor consumo de combustible
- Uso ineficiente de los recursos policiales
- Asignación incorrecta de patrullas disponibles



| OBJETIVOS DEL PROYECTO



General

Optimizar la asignación de rutas y patrullas mediante programación matemática.



GIS

Integrar datos viales reales de OSM para modelar grafos urbanos.



Modelado

Implementar un modelo de Programación Entera Binaria en Pyomo.

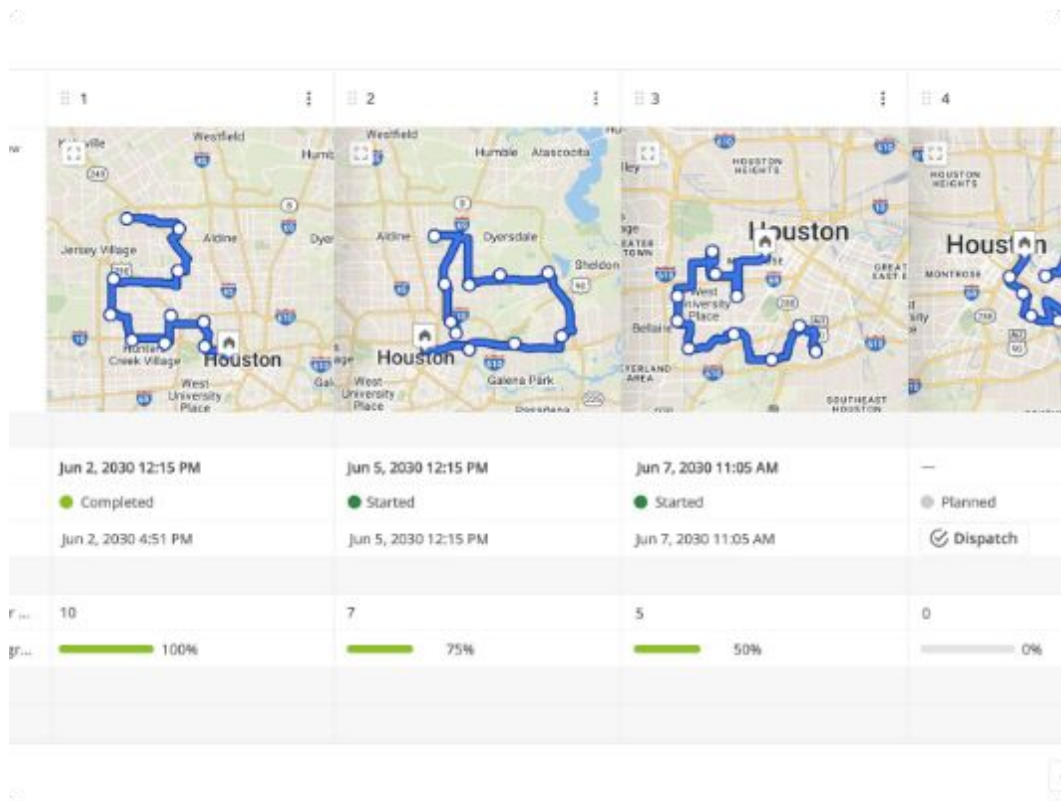


Eficiencia

Minimizar costos totales de operación y respuesta.

| RUTAS CANDIDATAS

Cada calle del grafo posee un peso asociado en nuestro caso el peso corresponde a la longitud de la calle en metros. El algoritmo utiliza estos pesos para encontrar la ruta de menor costo entre el origen y el destino y el sistema genera múltiples alternativas basadas en diferentes pesos:



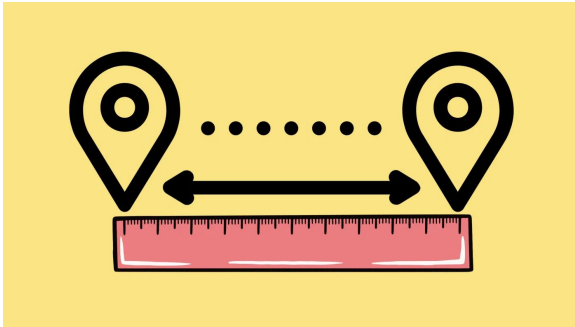
Ruta Corta (Distancia)

Ruta Rápida (Tiempo)

Ruta Ecológica (Consumo)

| Calculo de Distancia, Combustible y Tiempo

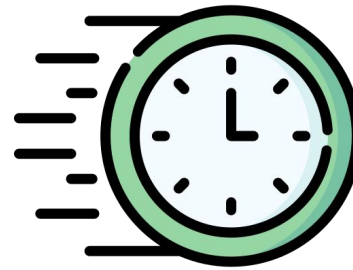
Distancia



- El usuario selecciona un punto A y un punto B.
- OSMnx obtiene la red de calles de Santa Cruz desde OpenStreetMap.
- NetworkX calcula la ruta más corta entre ambos puntos sobre las calles reales.
- Se suman las longitudes de todos los segmentos de la ruta.

$$Distancia = \sum_{i=1}^n Longitud\ del\ tramo_i$$

Tiempo



- Se utiliza la distancia total obtenida anteriormente.
- Se asume una velocidad promedio de circulación urbana de 35km/h

$$Tiempo = \frac{Distancia}{Velocidad\ Promedio}$$

Combustible



- Se toma la distancia calculada.
- Se asume un consumo promedio de la patrulla de 0.10 L/km.

$$Combustible = Distancia \times 0.10$$

| Flujo de Datos del Sistema de Información Geográfica



Input

Coordenadas de origen/destino
seleccionadas.



Extracción

Descarga de red vial vía OSMnx.



Procesamiento

Cálculo de pesos y rutas candidatas
con NetworkX.



Output

JSON de parámetros enviado al
motor Pyomo.

| VARIABLES DE DECISIÓN

Se define un conjunto de variables binarias que representan la elección óptima del sistema:

$$x_{r,p} \in \{0, 1\}$$

- **x = 1:** Si la ruta r es asignada a la patrulla p .
- **x = 0:** En caso contrario.

Sets del Modelo

R: Conjunto de rutas candidatas.

P: Conjunto de unidades de patrulla disponibles.

Parámetros

dr=distancia de la ruta

tr=tiempo estimado de la ruta r

fr=combustible estimado de la ruta r

trr=factor de tráfico de la ruta r

lp=carga de la patrulla p

| FUNCIÓN OBJETIVO

Buscamos minimizar el costo total de la operación:

$$\min Z = \sum_{r \in R} \sum_{p \in P} C_{r,p} x_{r,p}$$

Donde el costo unitario se calcula como: $C_{r,p}$

$$C_{r,p} = 1.0 d_r + 0.4 t_r + 8.0 f_r + 2.0 tr_r + 10 l_p$$

Se minimiza el costo total considerando distancia, tiempo, combustible, tráfico y carga de la patrulla.

| RESTRICCIONES DEL MODELO

Unicidad de Ruta

Garantiza que únicamente se seleccione una ruta y una patrulla.

$$\sum_{r \in R} \sum_{p \in P} x_{r,p} = 1$$

| MODELADO CON PYOMO

Definición del modelo ConcreteModel()

Definición de conjuntos R y P

Definición de variables binarias $x[r, p]$

Definición de función objetivo

Definición de restricciones y resolución con GLPK

```
model = pyo.ConcreteModel()

model.R = pyo.Set(initialize=routes)
model.P = pyo.Set(initialize=patrols)

model.x = pyo.Var(model.R, model.P, domain=pyo.Binary)

def cost(route_id, patrol):
    route = route_data[route_id]
    distance_cost = route["distance_km"] * weight_distance
    time_cost = route["estimated_time_minutes"] * weight_time
    fuel_cost = route["estimated_fuel_liters"] * weight_fuel
    traffic_cost = route["traffic_factor"] * weight_traffic
    load_cost = patrol_load[patrol] * penalty_load
    return distance_cost + time_cost + fuel_cost + traffic_cost + load_cost

def objective_rule(model):
    return sum(cost(r, p) * model.x[r, p] for r in model.R for p in model.P)
model.objective = pyo.Objective(rule=objective_rule, sense=pyo.minimize)

def only_one_route_patrol_rule(model):
    return sum(model.x[r, p] for r in model.R for p in model.P) == 1
model.only_one_assignment = pyo.Constraint(rule=only_one_route_patrol_rule)

solver = pyo.SolverFactory("glpk")
result = solver.solve(model)
```

| CICLO DE UNA CONSULTA



1. Input

Click en mapa
(Leaflet).



2. Routing

Generación de Ruta más
corta y alternativas.



3. Optimization

Pyomo + GLPK calculan el
óptimo.



4. Resultado

Ruta trazada en la
interfaz.

iGracias!